

最优可调整大小数组

Robert E. Tarjan Uri Zwick

Department of Computer Science, Princeton University, NJ, USA

Blavatnik School of Computer Science, Tel Aviv University, Tel Aviv, Israel

arXiv:2211.11009v2 [cs.DS] 2023 年 5 月 29 日

2026 年 7 月 15 日

摘要

可调整大小数组 (resizable array) 是一种可以通过在其末端（或两端）添加或删除项来增长和收缩的数组，同时仍支持通过索引在常数时间内访问数组中存储的每个项。由于数组的大小（即其中项的数量）随时间变化，空间高效地维护可调整大小数组需要动态内存管理。标准的**加倍技术**允许仅使用 $O(N)$ 空间维护大小为 N 的数组，每次操作的摊销时间为 $O(1)$ ，甚至最坏情况时间也为 $O(1)$ 。Sitarski 和 Brodnik 等人描述了更好的解决方案，仅使用 $N + O(\sqrt{N})$ 空间维护大小为 N 的可调整大小数组，每次操作仍为 $O(1)$ 时间。Brodnik 等人给出了一个简单的证明，表明这是可能达到的最优结果。

我们区分了**存储**可调整大小数组和访问其项所需的空间，以及在增长或收缩数组时可能**临时**需要的空间。对于每个整数 $r \geq 2$ ，我们证明 $N + O(N^{1/r})$ 空间足以**存储和访问**大小为 N 的数组，如果在增长和收缩操作期间可以短暂使用 $N + O(N^{1-1/r})$ 空间。通过索引访问项需要 $O(1)$ 最坏情况时间，而增长和收缩操作需要 $O(r)$ 摊销时间。通过对**增长游戏**的精确分析，我们证明对于使用仅 $N + O(N^{1/r})$ 空间存储数组的广泛一类数据结构中的任何数据结构，增长操作的摊销成本为 $\Omega(r)$ ，即使只允许增长和访问操作。除非 $r = 2$ ，增长和收缩操作的时间不能做到最坏情况。

1 1 引言

数组和可调整大小数组可能是使用最广泛的数据结构。令人惊讶的是，我们发现在它们之上似乎并非一切都已经说尽。我们描述了可调整大小数组的简单新实现，这些实现在许多情况下比迄今为止提出的所有实现都更具内存效率。我们的视角是理论性的，但我们提出的实现可能具有一定的实际意义。

大小为 N 的**数组**是一个保存项序列 $a_0, a_1, \dots, a_{N-1}$ 的数据结构。对于任意 $0 \leq i < n$, 序列中的第 i 个项可以在常数时间内被检索或修改。数组自然地实现为 N 个内存字的连续块, 每个字保存一个项, 或指向一个项的指针。使用机器的随机访问能力, 项在常数时间内被检索或修改。这里定义的数组本质上是固定大小的。

可调整大小数组 (也称为**动态数组**或**动态表**) 是一种大小可以增加或减少的数组, 同时仍然允许在常数时间内检索或修改任意 i 的第 i 个项。减小数组的大小很容易。我们简单地不使用分配给数组的部分内存。然而, 这并不具备内存效率。增加数组的大小则更难, 因为紧挨着分配给数组的连续块之后的那个字可能正被用于不同的目的。因此, 维护可调整大小数组需要动态内存管理。

为简单起见, 我们在论文的大部分内容中假设可调整大小数组仅在其“远端”增长和收缩。因此, 当数组的大小从 N 增加到 $N+1$ 时, 一个新项 a_N 被添加到数组中。类似地, 当大小从 N 减小到 $N-1$ 时, 项 a_{N-1} 消失。这种可调整大小数组形成了栈的推广。我们所有的结果也扩展到可以在两端增长和收缩的可调整大小数组, 因此它们给出了双端队列 (deque) 的推广。这很容易得到: 可以轻松地使用两个单端可调整大小数组实现一个双端可调整大小数组。

可调整大小数组的标准教科书解决方案 (参见例如 Cormen 等人 [4, 第 17.4 节]) 是**加倍或几何扩展/收缩**技术。首先分配某个初始大小的数组。如果数组已满, 分配一个大小为原来两倍的新数组并将所有项复制进去。当数组的使用率低于四分之一时, 分配一个大小为原来一半的新数组并再次复制所有项进去。在这两种情况下, 旧数组使用的内存被释放并可用于其他目的。简单的摊销论证表明, 每次大小增加或减少的摊销成本为 $O(1)$ 。每个项仍然可以在 $O(1)$ 最坏情况时间内被访问和修改。此外, 通过标准的后台重建技术, 增加或减少操作的时间可以做到 $O(1)$ 最坏情况。

根据 Wikipedia [16], 几何扩展/收缩技术 (增长因子从 1.25 到 2 不等) 被用于实现 Java 的 ArrayList、Python 的 PyListObject、C++ STL 的 Vector 等。

加倍技术允许使用 $O(N)$ 空间存储大小为 N 的可调整大小数组。这很好, 并且在许多应用中可能已经足够。但这也意味着在不同时刻, 分配给数组的内存中可能有多达 50% 未被使用, 即使只考虑增长操作。更一般地, 如果使用 $1 + \delta$ 的增长因子, 则 $\frac{\delta}{1+\delta}$ 比例的内存可能被浪费。通过选择很小的 δ 值, 浪费的存储比例可以变得任意小, 但大小增加的摊销成本变为 $O(\frac{1+\delta}{\delta})$, 即越来越大。

是否可能仅使用 $N + o(N)$ 空间存储大小为 N 的可调整大小数组, 同时每次操作仍维持 $O(1)$ 时间? Sitarski [15] 和 Brodnik 等人 [2] 通过描述仅使用 $N + O(\sqrt{N})$ 空间的简单实现肯定地回答了这个问题。(详见第 3 节。) Brodnik 等人 [2] 还证明这在某种意义下是最优的。更具体地说, 他们证明任何可调整大小数组的实现有时必须使用 $N + \Omega(\sqrt{N})$ 空间。(我们在第 4 节回顾他们的简单论证。)

我们提议区分**存储**当前大小为 N 的可调整大小数组所需的空间量 (支持在 $O(1)$ 最坏情况时间内检索和修改项), 以及临时增加或减少数组大小 (在 $O(1)$ 摊销时间内) 可

能需要的空间。在许多设置中，拥有更紧凑的数组表示是更可取的，即使调整数组大小时需要临时更多的存储空间。例如，一个应用可能使用许多可调整大小数组，在任何给定时刻只有一个数组被调整大小。

我们”打破” Brodник 等人 [2] 下界的第一个实现是一个简单的实现，仅使用 $N + O(N^{1/3})$ 空间存储数组，同时临时使用 $N + O(N^{2/3})$ 空间来调整其大小。项可以在 $O(1)$ 最坏情况时间内被访问，增加或减少数组大小需要 $O(1)$ 摊销时间。此外，我们证明这本质上是**最优的**。任何仅使用 $N + O(N^{1/3})$ 空间存储数组的实现必须在调整大小期间的某些时刻使用 $N + \Omega(N^{2/3})$ 空间。

更一般地，我们证明对于每个 $r \geq 2$ ，可以使用 $N + O(N^{1/r})$ 空间存储可调整大小数组，同时仅需要 $N + O(N^{1-1/r})$ 空间来调整其大小。通过索引检索和修改项需要 $O(1)$ 最坏情况时间，而调整大小需要 $O(r)$ 摊销时间。由于不能使用后台重建， $O(r)$ 摊销时间的调整大小（我们证明这是最优的）不能做到最坏情况，除非 $r = 2$ ，在这种情况下实现变得非常类似于 Sitarski [15] 和 Brodник 等人 [2] 的实现。

本文的其余部分组织如下。第 2 节给出问题和所用计算模型的更精确定义。第 3 节描述先前工作，主要是 Sitarski [15] 和 Brodник 等人 [2] 的工作。第 4 节描述 Brodник 等人 [2] 的简单下界及我们对它的扩展。第 5 节给出我们简单的 $r = 3$ 解决方案，即存储使用 $N + O(N^{1/3})$ 空间、调整大小使用 $N + O(N^{2/3})$ 空间。第 6 节描述对于任意 $r \geq 2$ ，一个使用 $N + O(rN^{1/r})$ 空间存储数组、在调整大小操作期间仅使用 $N + O(N^{1-1/r})$ 空间的实现。增长和收缩操作的摊销成本为 $O(r)$ 。第 7 节给出一个简单的变换，可用于调整第 6 节的实现，并将存储数组所需的空间减少到 $N + O(N^{1/r})$ ，同时维持访问的常数时间和增长与收缩操作的 $O(r)$ 摊销时间。（注意这仅在 $r = r(N)$ 随 N 增长时才有意义。）为了证明调整大小操作的 $O(r)$ 摊销时间是最优的（至少对于我们称为**标准的**广泛一类数据结构），我们在第 8 节引入一个抽象的增长游戏并对其进行完全分析。在第 9 节中，我们依赖增长游戏的分析来获得增长操作摊销成本的 $\Omega(r)$ 下界（如果仅能使用 $N + O(N^{1/r})$ 空间存储数组）。我们在第 10 节以一些总结性评论结束。

2 2 可调整大小数组

可调整大小数组是一个抽象数据类型，支持以下操作：

- $A \leftarrow \text{Array}()$ — 创建并返回一个初始为空的数组。
- $A.\text{Length}()$ — 返回数组 A 的当前长度。
- $A.\text{Get}(i)$ — 返回数组 A 中的第 i 个项 a_i 。假设 $0 \leq i < A.\text{Length}()$ 。
- $A.\text{Set}(i, a)$ — 将数组 A 中的第 i 个项改为 a 。假设 $0 \leq i < A.\text{Length}()$ 。
- $A.\text{Grow}(a)$ — 将数组 A 的长度增加 1，并将新的最后一项设置为 a 。
- $A.\text{Shrink}()$ — 将数组 A 的长度减小 1，丢弃其最后一项。

注意项只能从数组的末端添加或移除。允许从数组的任意位置添加或移除项，并相应更新更高索引项的索引，会使问题变得困难得多，即使不需要空间效率。所有操作可

以同时获得的最佳时间界是 $O(\log N / \log \log N)$ ，其中 N 是数组的大小 (Dietz [5])，并且这是紧的，这来自 Fredman 和 Saks [6] 的结果。对于任意 $r \geq 1$ ，可以以 $O(r)$ 最坏情况时间实现访问操作，但插入和删除需要 $O(N^{1/r})$ 摊销时间。(参见例如 Goodrich 和 Kloss [8]、Joannou 和 Raman [9]、Katajainen [11] 和 Bille 等人 [1]。) 我们再次强调，我们只允许在数组的末端添加或移除项。

我们假设内存管理系统允许我们分配和释放任意大小的固定长度数组。(作为例子，考虑 C 或 C++ 的 `malloc` 和 `free` 函数。) 为简单起见，我们假设每次这样的调用只需要常数时间。然而，如果分配或释放大小为 N 的固定长度数组需要 $O(N)$ 时间，我们所有的摊销界仍然成立。我们不考虑内存管理系统的内部工作机制。

可调整大小数组的实现必须使用一系列动态分配的固定长度数组，有时称为**块**。通常，这些块中的每一个要么是**数据块**，包含可调整大小数组的项（每个项在一个单独的字中）；要么是**索引块**，包含指向其他块的指针（每个指针在一个单独的字中）；要么是**辅助块**，包含数据结构使用的辅助信息，如各个块的长度。本文中给出的所有数据结构都是这种形式。如果可调整大小数组当前大小为 N ，则所有数据块的总大小必须至少为 N 。(另见第 4 节中定理 4.1 证明之后的讨论。) 每个块（除了被称为**主索引块**的一个索引块外）必须由某个索引块中的一个指针指示，否则该块将不可访问。数据结构使用的空间是分配的所有块的长度之和。

为简化本文中讨论的各种数据结构的讨论，我们引入以下定义。

定义 2.1 ($(s(N), t(N))$ -实现)。令 $s(N), t(N)$ 为两个非递减函数。称一个可调整大小数组数据结构为 $(s(N), t(N))$ -**实现**，如果它使用最多 $N + s(N)$ 空间存储大小为 N 的数组，并在大小为 N 的数组上的增长或收缩操作期间使用最多 $N + t(N)$ 空间。

3 3 先前工作

设计空间高效的可调整大小数组实现的问题当然属于简洁数据结构的领域。参见例如 Raman 等人 [14]、Munro 和 Srinivasa [13] 及其中的参考文献。但似乎只有少数几篇论文（我们接下来将描述）涉及了此处定义的基本可调整大小数组问题。

3.1 3.1 基本数据结构

可调整大小数组的基本数据结构仅使用单个固定大小数组来实现可调整大小数组。当固定大小数组已满时，基本数据结构需要分配一个更大的数组，将所有项从旧数组复制到新数组，添加新项，然后释放旧数组。基本数据结构可以自由选择新固定大小数组的大小。在一些收缩操作之后，基本数据结构可能认为固定大小数组太空了，此时它可以分配一个更小的数组并将所有项复制进去，释放旧数组。

最节省空间的基本数据结构保持一个大小正好等于可调整大小数组大小的固定大小数组。每次增长和收缩操作之后都需要分配一个新数组。存储大小为 N 的数组所使

用的空间是 $N + O(1)$ 。(我们需要存储数组的长度和指向固定大小数组的指针。)然而,增长和收缩操作的(摊销)成本为 $\Omega(N)$ 。此外,在实现增长或收缩操作时,数据结构临时需要 $2N + O(1)$ 空间,因为两个数组需要同时存储。这样的数据结构只应在增长或收缩操作很罕见时使用。

更实用的基本数据结构使用引言中讨论的几何扩展/收缩技术。选择一个固定参数 $\delta > 0$ 。当分配的数组已满时,分配一个大小为 $(1 + \delta)N$ 的新数组,将所有项复制进去并释放旧数组。当大小为 N 的分配数组仅包含 $N/(1 + \delta)^2$ 个项时,分配一个大小为 $N/(1 + \delta)$ 的新数组,将所有项复制进去并释放旧数组。简单计算表明,增长操作的摊销成本为 $\frac{1+\delta}{\delta}$, 收缩操作的摊销成本为 $\frac{1}{\delta}$ 。¹ δ 可以为许多实际情况提供满意的解决方案。

¹ $(1+\delta)N_{\delta N}$, 而收缩操作的摊销成本为 $\frac{N}{(1+\delta)^2} / \left(\frac{N}{1+\delta} - \frac{N}{(1+\delta)^2} \right)$ 。注意在每次调整大小操作之后,若 N 是数组中当前的项数,则数组的大小为 $(1 + \delta)N$ 。

3.2 3.2 Sitarski 的数据结构

Sitarski [15] (另见 [17]) 描述了一个漂亮且简单的可调整大小数组实现,他称之为**哈希数组树** (Hashed Array Tree, HAT)。这个名字有点不恰当,因为没有涉及哈希,而且数据结构使用的”树”实际上是一个列表。

HAT 数据结构维护一个大小为 B 的索引块 I , 其中 $\sqrt{N} \leq B < 4\sqrt{N}$, 以及 $\lceil N/B \rceil$ 或 $\lceil N/B \rceil + 1$ 个数据块, 每个大小为 B , 如图 1 所示。数组中当前的 N 个项按顺序存储在前 $\lceil N/B \rceil$ 个数据块中。如果 N 不能被 B 整除, 第 $\lceil N/B \rceil$ 个数据块只是部分满的。第 $(\lceil N/B \rceil + 1)$ 个数据块(如果存在)是空的。索引块中的第 i 个条目包含指向第 i 个数据块的指针。

该数据结构仅使用 $N + 3B + 2 = N + O(\sqrt{N})$ 个字的内存。($3B$ 项对应索引块的大小和两个可能为空的数据块。)第 i 个项存储在第 $\lfloor i/B \rfloor$ 个数据块的第 $(i \bmod B)$ 个位置, 因此访问或修改它需要 $O(1)$ 最坏情况时间。如果 B 被选为 2 的幂, 计算这些索引特别容易。

如果 $N < B$, 或等价地 $N < B^2$, 则扩展数组很简单。如果第 $\lceil N/B \rceil$ 个数据块不满, 新项被添加为该数据块的最后一个项。如果第 $\lceil N/B \rceil$ 个数据块已满但第 $(\lceil N/B \rceil + 1)$ 个数据块已分配, 新项成为该数组的第一个项。否则, 分配一个大小为 B 的新数据块, 新项成为其第一个项。如果假设分配新数组需要 $O(1)$ 时间, 扩展需要 $O(1)$ 最坏情况时间; 如果假设分配新数组需要 $O(B)$ 时间, 扩展需要 $O(1)$ 摊销时间。

当数据结构满时, 即 $N = B^2$, 数据结构以加倍后的 B 重建。如果 B 最初被选为 2 的幂, 它将保持为 2 的幂, 从而简化索引操作。摊销成本仍为 $O(1)$, 因为重建操作的 $O(N)$ 成本可以记到自上次重建以来必然发生的最后 $\frac{3}{4}N$ 次增长操作上。(注意如果 $B = \sqrt{N}$ 且 B 加倍, 则数据结构的容量从 N 增加到 $4N$ 。)

收缩操作的实现类似。为避免释放并立即重新分配一个数据块, 仅当最后两个数据

块都为空时才释放最后一个数据块。当 $B = 4\sqrt{N}$ ，或等价地 $N = B^2/16$ 时， B 的值减半并重建数据结构。增长和收缩操作的摊销成本仍为 $O(1)$ ，因为两次重建操作之间必须发生至少 $\Omega(N)$ 次操作。

重要的是要注意，重建操作可以在仅使用 $O(\sqrt{N})$ 额外存储的情况下进行。新旧数据块逐个分配和释放。

可以通过维护两种可能大小的数据块并进行后台重建来去摊销化 HAT 数据结构，但得到的数据结构会变得更加复杂。下面描述一个具有最坏情况界的更简单数据结构。

3.3 3.3 Brodnik 等人的数据结构

Brodnik 等人 [2]（显然不知道 Sitarski [15] 的 HAT 数据结构）描述了一个优雅的数据结构，仅使用 $N + O(\sqrt{N})$ 空间，并为增长和收缩操作实现 $O(1)$ 最坏情况时间界（假设内存分配需要 $O(1)$ 最坏情况时间）。该数据结构吸引人的特点是无需重建操作，且数据项永不被移动。

Brodnik 等人 [2] 的数据结构有两个变体，在图 2 中示意性地描述。

在第一个变体（如图 2(a) 所示）中，数组的项按顺序存储在大小为 $1, 2, 3, \dots$ 的数据块中。由于 $1 + 2 + \dots + k = \frac{k(k+1)}{2} \approx k^2/2$ ，需要大约 $\sqrt{2N}$ 个块来存储 N 个项。更精确地说，这个数是 $k = \left\lceil \frac{\sqrt{8N+1}-1}{2} \right\rceil$ 。最后一个数据块可能只是部分填充的。一个额外的第 $(k+1)$ 个数据块可能被分配，此时它是完全空的。（这个最后的数据块曾经包含后来被移除的项。）一个大小为 $B = \Theta(\sqrt{N})$ 的索引块存储指向数据块的指针。该索引块实际上是使用朴素方法实现的可调整大小数组。

增长和收缩数组是相当简单的操作。一个新项被添加为部分填充块的最后一项（如果有的话），或者空块的第一项（如果有的话）。否则，分配一个新数据块，新项成为其第一项。指向新数据块的指针被添加到索引块。如果索引块已满，分配一个更大的索引块并将所有指针复制过去。旧索引块被释放。为获得 $O(1)$ 最坏情况时间界，此复制应在“后台”进行。收缩操作类似。

数组中的第 i 个项可以在 $O(1)$ 最坏情况时间内被访问，但计算此项所在数据块的索引需要计算平方根。更精确地说，第 i 个项存储在第 ℓ 个数据块中，其中 $\ell = \left\lceil \frac{\sqrt{8i+1}-1}{2} \right\rceil$ ，并且它是该块中的第 $(i - \frac{\ell(\ell-1)}{2})$ 个项。

为避免计算平方根，Brodnik 等人 [2] 描述了其数据结构的一个变体，如图 2(b) 所示。这里每个数据块的大小是 2 的幂。更精确地说，对于 $k = 0, 1, \dots$ ，存在一个虚拟的超级块，由 $2^{\lceil k/2 \rceil}$ 个大小为 $2^{\lfloor k/2 \rfloor}$ 的块组成。计算第 i 个项所在的超级块及其在此超级块中的位置现在可以使用简单的移位操作完成。具体细节参见 [2]。²

² [2]

4 4 Brodnik 等人的下界及其扩展

我们描述 Brodnik 等人 [2] 下界的一个稍作修改的版本。

定理 4.1. 维护可调整大小数组的任何数据结构必须在某些时刻使用 $N + \Omega(\sqrt{N})$ 空间, 其中 N 是数组的当前长度, 即使只执行增长和访问操作。

证明. 令 k 为在 N 次增长操作序列之后数据结构使用的连续内存块的数量。这些块必须包含所有 N 个项, 否则并非所有项都能被访问。因此, 块的总大小至少为 N 。(见证明后的讨论。) 数据结构还必须维护指向每个这样的块的指针, 否则它将无法访问该块。因此, 如果 $k \geq \sqrt{N}$, 数据结构使用至少 $N + k \geq N + \sqrt{N}$ 空间。

如果 $k < \sqrt{N}$, 则最大分配的块的大小必须为 $\ell > N/k > \sqrt{N}$ 。此块在数组包含 $N' \leq N$ 个项时被分配。在此块刚被分配且其大小包含在数据结构使用的总空间中之后, N' 个项必须存储在其他块中。因此, 该时刻使用的总空间为 $N' + \ell > N' + \sqrt{N}$ 。

下界假设存储 N 个项需要 N 个字的内存。如果每个内存字是 w 位的且一个项是任意的 w 位串, 此假设成立。下界还假设存储 k 个指针需要 k 个字的内存。如果假设内存大小为 2^w , 这成立。如果内存大小仅为 2^{cw} ($0 < c < 1$), 则仅需 cw 位表示一个指针, k 个指针可以用大约 ck 个字表示。这改变了额外空间的下界, 但仅相差一个常数因子。

Brodnik 等人 [2] 的下界可以很容易地扩展为一个分别考虑存储可调整大小数组使用的空间和增长或收缩时临时需要的空间的下界。回顾定义 2.1, 可调整大小数组的 $(s(N), t(N))$ -实现是仅使用 $N + s(N)$ 空间存储大小为 N 的数组且在调整大小操作期间使用最多 $N + t(N)$ 空间的实现。

定理 4.2. 任何可调整大小数组的 $(s(N), t(N))$ -实现必须有 $s(N) \cdot t(N) = \Omega(N)$, 即使只支持增长和访问操作。

证明. 考虑在 N 次增长操作之后数据结构的状态。所有 N 个项必须分布在若干连续内存块中。由于必须为每个这样的块维护指针, 且使用的额外空间最多为 $s(N)$, 块的数量最多为 $s(N)$ 。因此, 至少有一个块 (记为 B) 的大小至少为 $N/s(N)$ 。令 $N' \leq N$ 为分配 B 的增长操作的索引。在 B 刚被分配时, 它不包含任何项。因此, 此时数据结构使用的临时额外空间至少为 $N'/s(N')$ 。因此, $t(N) \geq t(N') \geq N'/s(N')$, 如所需。(注意这里使用了 $t(N)$ 的单调性。)

推论 4.3. 对于任意整数 $r \geq 1$, 任何仅使用 $N + O(N^{1/r})$ 空间存储大小为 N 的可调整大小数组的数据结构, 必须偶尔在增长操作期间使用 $N + \Omega(N^{1-1/r})$ 空间, 即使只执行增长和访问操作。

在接下来的章节中, 我们将证明这个下界是紧的。

5 简单的 $(O(N^{1/3}), O(N^{2/3}))$ -实现

在本节中，我们描述简单的 $(O(N^{1/3}), O(N^{2/3}))$ -实现，具有 $O(1)$ 最坏情况访问时间和 $O(1)$ 增长或收缩操作摊销时间。

基本思想很简单。首先考虑 HAT 数据结构，但数据块的大小约为 $N^{2/3}$ 。索引块的长度约为 $N^{1/3}$ 。不幸的是，由于最后一个数据块可能几乎为空，数据结构使用的总空间为 $N + O(N^{2/3})$ 。为了解决这个问题，我们使用一个 HAT 数据结构来处理大小为 $N^{2/3}$ 的最后一个部分填充的数据块。存储数组的额外空间现在仅为 $O((N^{2/3})^{1/2}) = O(N^{1/3})$ 。当最后一个数据块的大小达到 $N^{2/3}$ 时，它被复制到一个大小为 $N^{2/3}$ 的新数据块中。在此复制操作期间，使用了 $\Theta(N^{2/3})$ 额外空间。

也很容易基于 Brodnik 等人 [2] 的数据结构得到一个类似的数据结构。该数据结构使用大小为 $1, 4, \dots, i^2, \dots, k^2$ 然后 $1, 2, 3, \dots$ 的块。

第一个数据结构的完整、非递归描述如图 3 所示。数据结构维护一个参数 B ，使得 $N^{1/3} \leq B \leq 4N^{1/3}$ 。它使用大约 N/B^2 个大小为 B^2 的数据块（我们称为**大块**），以及最多 $2B$ 个大小为 B 的数据块（我们称为**小块**）。现在有两个索引块，一个用于大数据块，一个用于小数据块。大块总是满的。最多一个小块是部分填充的，也最多一个小块是空的。当 $2B$ 个小块满了时，其中 B 个被复制到一个新分配的大块中。当没有非空的小块时，一个大块被分割为 B 个新分配的小块。很容易看出所有操作仅需要 $O(1)$ 时间（增长和收缩摊销，访问和修改最坏情况）。下一节给出形式化证明。

6 对于任意 $r \geq 2$ 的 $(O(rN^{1/r}), O(N^{1-1/r}))$ -实现

推广第 5 节的构造，我们为每个 $r \geq 2$ 获得一个 $(O(rN^{1/r}), O(N^{1-1/r}))$ -实现。可以如第 5 节开头那样以递归方式得到这样的实现。在本节中，我们获得这些实现的非递归版本。

对于给定的 $r \geq 2$ ，数据结构维护一个参数 B ，使得 $N^{1/r} \leq B < 4N^{1/r}$ 。它将项保存在大小为 B 的幂的块中，即 $B, B^2, \dots, B^{r-1}$ 。（见图 4。）

如果只支持增长操作（不支持收缩），我们可以维持这样一个不变式：每种大小的块总是最多有 B 个。（为允许高效的收缩操作，我们很快将放松此条件，允许每种大小最多有 $2B$ 个块，如图 4 所示。）大小为 $B^2, \dots, B^{r-1}$ 的块总是满的。所有长度为 B 的块（可能除最后一个外）也是满的。如果最后一个大小为 B 的块不完全满，则增长操作很容易。如果它满了，且大小 B 的块少于 B 个，则分配一个新的大小为 B 的块，新项被放入其中。如果有 B 个大小为 B^i 的满块，则分配一个新的大小为 B^{i+1} 的块，这 B 个块中的项按适当顺序复制到新块中，然后这 B 个大小为 B^i 的块被释放。然后分配一个新的大小为 B 的块，新项被放入其中。注意这模仿了 B 进制计数器的操作。

当 $N = B^r$ 时， B 的值加倍，整个数据结构被重建。（我们可以将重建推迟到数据结构完全满时，即 $N = B^r + B^{r-1} + \dots + 1 = \frac{B^{r+1}-1}{B-1}$ ，但这不会带来太多好处。为简单

起见,我们在 $N = B^r$ 时重建。)

数据结构维护 $r - 1$ 个索引块,每种可能的块大小一个。存储包含 N 个项的数组所用的总空间最多为 $N + (2r - 1) + 2(r - 1)B + B = N + O(rN^{1/r})$ 。实现使用的最大额外空间量为 $O(N^{1-1/r})$,即当分配一个大小为 B^{r-1} 的新块时。

很容易看出每个项可以在 $O(r)$ 最坏情况时间内被访问。通过更多的关注,如我们在第 6.3 节中所展示的,这可以降低到 $O(1)$ 最坏情况时间。

增长操作的摊销成本为 $O(r)$,因为每个项首先被放入大小为 B 的块中,然后最终被移动到大小为 B^i 的块中 ($i = 2, \dots, r - 1$)。完全重建仅给增长操作的摊销成本增加一个常数,因为两次重建操作之间数据结构中的总项数以某个常数因子增长。

允许收缩操作同时维持每次增长和收缩操作的 $O(r)$ 摊销成本并不困难。一个标准解决方案是使用**冗余 B 进制计数器**。我们每种大小最多允许 $2B$ 个块,而不仅是 B 。当有 $2B$ 个大小为 B^i 的满块时 ($1 \leq i < r - 1$),分配一个新的大小为 B^{i+1} 的块,前 B 个大小为 B^i 的块被复制进去然后释放。如果收缩操作发生在没有大小为 B 的块时,一个大小为 B^i 的块 (对于最小的可能的 i) 被分割为 $B - 1$ 个大小为 $B^{i-1}, \dots, B^2$ 的块和 B 个大小为 B 的块。我们在第 6.2 节中证明增长和收缩操作的摊销成本再次为 $O(r)$ 。Kaplan 等人 [10] 在不同的背景下使用了类似的冗余 B 进制计数器。

6.1 6.1 实现细节

增长和收缩操作的伪代码如图 5 所示。如图 4 一样,我们假设 n_i (对于 $i \in [r - 1]$) 是大小为 B^i 的数据块的数量。我们用 $A[i][0], \dots, A[i][n_i - 1]$ 表示这些块。如果 $n_1 > 0$,我们令 n_0 为 $A[1][n_1 - 1]$ (最后一个大小为 B 的数据块) 中的项数。使用这种记号, $A[i]$ (对于 $1 \leq i \leq r - 1$) 是第 i 个索引块,其大小为 $2B$,而 A 是指向索引块的指针块,大小为 r 。每个块中的索引从 0 开始。(为方便起见,我们将单元 $A[0]$ 留空。)数 $n_0, n_1, \dots, n_{r-1}$ 保存在一个大小为 r 的块 n 中。(为简单起见,我们写 n_i 而非 $n[i]$ 。)块 A 和 $A[i]$ (对于 $1 \leq i \leq r - 1$) 仅在重建操作期间分配和释放。

伪代码假设存在一个过程 $\text{Allocate}(\ell)$ 分配一个大小为 ℓ 的新块并返回指向新分配块的指针,以及一个过程 $\text{Deallocate}(X)$ 释放先前分配的块 X 。它还假设 $\text{Copy}(X, x, Y, y, \ell)$ 将 $X[x..x + \ell - 1]$ 中的项复制到 $Y[y..y + \ell - 1]$,其中 $X[x..x + \ell - 1]$ 是 $X[x], X[x + 1], \dots, X[x + \ell - 1]$ 的简写。(换句话说,它执行赋值 $Y[y + i] \leftarrow X[x + i]$,对于 $i = 0, 1, \dots, \ell - 1$ 。)过程 $\text{Rebuild}(B)$ (未显示) 以新的参数 B (通常是 $2B$ 或 $B/2$) 重建数据结构。

下面给出伪代码:

```

Grow(a):
    if N = B^r :
        Combine-Blocks():
            k ← min{ i ∈ [r-1] | n_i < 2B }
            if k = 0 : error

```

```

    Rebuild(2B)
else if n_1 = 2B and n_0 = B :
    Combine-Blocks()
else if n_1 = 0 or n_0 = B :
    A[1][n_1] ← Allocate(B)
    n_1 ← n_1 + 1
    n_0 ← 0

A[1][n_1-1][n_0] ← a
n_0 ← n_0 + 1
N ← N + 1

Shrink():
    if N = (B/4)^r :
        Rebuild(B/2)
    else if n_1 = 0 :
        Split-Blocks()

    n_0 ← n_0 - 1
    N ← N - 1

    if n_0 = 0 :
        Deallocate(A[1][n_1-1])
        n_0 ← B
        n_1 ← n_1 - 1

    for i ← k-1 downto 1 :
        A[i+1][n_{i+1}] ← Allocate(B^{i+1})
        for j ← 0 to B-1 :
            Copy(A[i][j], 0, A[i+1][n_{i+1}], jB^i)
            Deallocate(A[i][j])
        A[i][j] ← A[i][j+B] // 移位索引
        n_i ← B
        n_{i+1} ← n_{i+1} + 1

Split-Blocks():
    k ← min{ i ∈ [r-1] | n_i > 0 }
    if k = : error
    for i ← k-1 downto 1 :
        n_{i+1} ← n_{i+1} - 1
        for j ← 0 to B-1 :
            A[i][j] ← Allocate(B^i)
            Copy(A[i+1][n_{i+1}], jB^i, A[i][j], 0)
            Deallocate(A[i+1][n_{i+1}])

```

图 5: $(O(rN^{1/r}), O(N^{1-1/r}))$ -实现的增长和收缩操作伪代码。

Grow(a) 的工作方式如下。如果 $N = B^r$ ，数据结构以 $B \leftarrow 2B$ 重建。如果第一层完全满了，即 $n_1 = 2B$ 且 $n_0 = B$ ，则调用 Combine-Blocks()。该过程找到最小的索引 $k \in [r-1]$ 使得 $n_k < 2B$ 。（这样的索引必定存在。）对于 $i = k-1, k-2, \dots, 1$ ，它将 B 个大小为 B^i 的块合并为一个大小为 B^{i+1} 的新块，根据需要分配和释放块。更具体地说，为维持顺序，前 B 个大小为 B^i 的块（即 $A[i][0], \dots, A[i][B-1]$ ）被合并形成一个新的最后一块 $A[i+1][n_{i+1}]$ ，大小为 B^{i+1} 。块 $A[i][B], \dots, A[i][2B-1]$ 变为

$A[i][0], \dots, A[i][B-1]$ 。(这涉及指针更改。没有项被移动。通过将 $A[i]$ 视为循环数组可以避免这些指针更改。) 最后, 大小为 B^i 的块的数量被设为 B , 大小为 B^{i+1} 的块的数量加一, 即 $n_i \leftarrow B$ 且 $n_{i+1} \leftarrow n_{i+1} + 1$ 。

如果 Rebuild 或 Combine-Blocks 未被调用, Grow(a) 检查大小为 B 的块中是否有空位。如果没有, 即如果 $n_1 = 0$ (没有大小为 B 的块), 或者 $n_1 > 0$ 但 $n_0 = B$ (所有大小为 B 的块都满了), 则分配一个新的空大小为 B 的块 $A[1][n_1]$, n_1 递增, n_0 设为 0。最后, 在所有情况下, 新项 a 被放入空或部分填充的 B 块的第一个空位, 即 $A[1][n_1 - 1][n_0] \leftarrow a$, 且 n_0 和 N 递增。

Shrink 类似。如果 $N = (B/4)^r$, 数据结构以 $B \leftarrow B/2$ 重建。否则, 如果 $n_1 = 0$, 即没有大小为 B 的块, Split-Blocks 找到最小的 $k \in [r-1]$ 使得 $n_i > 0$ 。最后一个大小为 B^k 的块被分割为 $B-1$ 个大小为 $B^{k-1}, \dots, B^2$ 的块和 B 个大小为 B 的块。最后, 在所有情况下, 最后一个大小为 B 的块中的最后一项被丢弃, n_0 和 N 递减。如果 $n_0 = 0$, 空的大小为 B 的块被释放, n_1 递减且 n_1 设为 B 。(实际上, 延迟释放最后一个大小为 B 的块更好, 如第 3.2 节中 $r = 2$ 和第 5 节中 $r = 3$ 的做法。相应地更改伪代码并不困难。)

定理 6.1。 该实现是可调整大小数组的一个 $(O(rN^{1/r}), O(N^{1-1/r}))$ -实现。增长和收缩操作的摊销成本为 $O(r)$ 。访问操作可以在 $O(1)$ 最坏情况时间内支持。

证明。 实现的正确性以及 $O(rN^{1/r})$ 和 $O(N^{1-1/r})$ 的空间界由上文的讨论得出。增长和收缩操作的摊销成本的 $O(r)$ 界由下面的引理 6.3 证明。在 $O(1)$ 最坏情况时间内实现访问操作的方法由下面的引理 6.4 的证明描述。

通过令 $r = \log N$, 我们得到以下有趣的推论:

推论 6.2。 存在一个可调整大小数组实现, 使用 $N + O(\log N)$ 空间存储大小为 N 的数组, 在 $O(\log N)$ 摊销时间内支持增长和收缩操作, 并在 $O(1)$ 最坏情况时间内支持访问操作。

6.2 6.2 增长和收缩操作的摊销成本

引理 6.3。 $(O(rN^{1/r}), O(N^{1-1/r}))$ -实现中增长和收缩操作的摊销成本为 $O(r)$ 。

证明。 我们将 Grow 或 Shrink 操作的成本定义为其执行的项赋值次数, 并获得关于这些成本的 $O(r)$ 摊销界。由于 Grow 或 Shrink 操作执行的总操作数与此定义的成本加一成正比, 这足以证明引理。(加一是为了处理不执行任何项赋值的 Shrink 操作。)

Grow 操作的成本为 1, 加上 Combine-Blocks 的成本 (如果被调用)。类似地, Shrink 操作的成本为 0, 加上 Split-Blocks 的成本 (如果被调用)。(大多数 Shrink 操作仅减少 N 和 n_0 , 不执行任何项赋值。) 如果 Combine-Blocks 操作创建一个大小为 B^k 的块, 则其成本为 $B(B + B^2 + \dots + B^{k-1}) \leq 2B^k$ (假设 $B \geq 2$)。如果 Split-Blocks 操作分割一个大小为 B^k 的块, 则其成本为 B^k 。

当前在第 i 层 (即在大小为 B^i 的数据块中) 的项被赋予 $r-i$ 个信用单位。每个信

用单位可以支付一次项赋值。Grow 操作的成本（不包括 Combine-Blocks 的成本，如果被调用）为 1。赋予新项的信用为 $r - 1$ （因为它总是被放在第 1 层），因此 Grow 操作的摊销成本（不包括 Combine-Blocks 的成本）为 r 。

Combine-Blocks 操作的摊销成本为 0，因为每次项赋值将一个项从第 i 层移动到第 $i + 1$ 层，这使该项的信用减少 1，从而支付该赋值。因此，Grow 的摊销成本最多为 r 。

为处理 Shrink 操作，我们也给层赋予信用。每次 Shrink 操作向每层添加 3 个信用单位。因此，Shrink 的摊销成本（不包括 Split-Blocks 的成本）最多为 $3r$ 。

接下来我们证明 Split-Blocks 操作的摊销成本最多为 0。我们首先注意，在大小为 B^k 的块连续两次分割之间，必须至少执行 B^k 次 Shrink 操作。实际上，在这样的分割之后，第 1 到 $k - 1$ 层恰好包含 B^k 个项。此外，在创建一个新的大小为 B^k 的块的 Combine-Blocks 操作之后，第 1 到 $k - 1$ 层也包含至少 B^k 个项。因此，在第 k 层的 Split-Blocks 操作与第 k 层的上一次 Combine-Blocks 或 Split-Blocks 操作之间，必须至少有 B^k 次 Shrink 操作。类似地，在第 k 层的第一个 Split-Blocks 操作之前，必须至少有 B^k 次 Shrink 操作。因此，当第 k 层发生 Split-Blocks 时，该层已经积累了至少 $3B^k$ 个未使用的信用单位。

第 k 层 Split-Blocks 操作的**实际成本**为 B^k 。需要添加到从第 k 层移动到更小层的项的额外信用总量为

$$B(k-1) + (B-1) \sum_{i=1}^{k-1} (k-i)B^i \leq B \sum_{i=1}^{k-1} (k-i)B^i = B^{k+1} \sum_{j \geq 1} jB^{-j} = \frac{B^{k+2}}{(B-1)^2} \leq 2B^k,$$

其中最后一个不等式假设 $B \geq 4$ 。因此，第 k 层的 $3B^k$ 个信用单位足以覆盖 Split-Blocks 操作的成本，因此其摊销成本最多为 0。

最后，我们需要计入重建的成本。现在每次 Grow 或 Shrink 操作向整个数据结构添加 $2r$ 个信用单位。（这当然使 Grow 和 Shrink 的摊销成本增加 $2r$ 。）在加倍 B 的 Rebuild 操作与上一次 Rebuild 操作之间的 Grow 操作次数至少为 $B^r - (B/2)^r = N(1 - 2^{-r}) \geq N/2$ 。在减半 B 的 Rebuild 操作与上一次重建操作之间的 Shrink 操作次数至少为 $(B/2)^r - (B/4)^r = (2^r - 1)N \geq N$ 。因此，当 Rebuild 即将发生时，数据结构已经积累了至少 rN 个未使用的信用单位。Rebuild 的**实际成本**恰好为 N 。需要赋予项的总信用量最多为 $(r-1)N$ 。因此，Rebuild 的摊销成本为 0。

上面给出的 Rebuild 摊销分析非常宽松。当 Rebuild 加倍 B 时，所有项的总信用实际上是**减少**的。因此，每次 Grow 操作只需要向数据结构添加 $1/(1 - 2^{-r}) \approx 2$ 个信用单位。类似地，每次 Shrink 只需要向数据结构添加 $r/(2^r - 1) \leq 1$ 个信用单位。这当然不改变 Grow 和 Shrink 操作的 $O(r)$ 摊销成本。

以上分析隐含地假设 r 是常数，即不随 N 变化。将分析适应于非常数 $r = r(N)$ 的情况是容易的。

6.3 6.3 在 $O(1)$ 最坏情况时间内访问和修改项

在本节中，我们证明第 6.1 节的 $(O(rN^{1/r}), O(N^{1-1/r}))$ -实现可以在 $O(1)$ 最坏情况时间内支持访问和修改操作。我们假设机器字 x 中最高有效设置位的索引 (记为 $\text{msb}(x)$) 可以在 $O(1)$ 时间内确定。(稍后进一步讨论此假设。)

引理 6.4。 $(O(rN^{1/r}), O(N^{1-1/r}))$ -实现可以在 $O(1)$ 最坏情况时间内支持访问和修改操作。

为简化访问操作的高效实现，我们对数据结构做一个小的改变。令 n_1 为大小为 B 的满数据块的数量 (即不包括那一个包含 $n_0 < B$ 个项的部分填充的大小为 B 的块，如果存在的话)。数 $n_2, \dots, n_{r-1}$ 仍然是大小为 $B^2, \dots, B^{r-1}$ 的满块的数量。(这些大小的所有块都是满的。) 我们还假设 B 是 2 的幂，即 $B = 2^b$ ($b > 0$ 为某个整数)。

注意 $N = \sum_{j=0}^{r-1} n_j B^j = (n_{r-1}, \dots, n_1, n_0)_B$ ，其中 $0 \leq n_j \leq 2B$ (对于 $0 \leq j < r$)。因此， $(n_{r-1}, \dots, n_1, n_0)$ 是 N 的一个可能冗余的 B 进制表示。我们将 $n_{r-1}, \dots, n_1, n_0$ 紧凑地存储在两个字中：

$$N'_0 = (n_{r-1}^0, \dots, n_1^0, n_0^0)_B, \quad N'_1 = (n_{r-1}^1, \dots, n_1^1, n_0^1)_B,$$

其中 $n_j = n_j^0 + Bn_j^1$ ， $0 \leq n_j^0 < B$ ， $0 \leq n_j^1 \leq 2$ (对于 $0 \leq j < r$)。注意 N'_a ($a = 0, 1$) 是每个 $n_{r-1}^a, \dots, n_1^a, n_0^a$ 的 b 位表示的拼接。每个 n_j^a 可以在常数时间内使用简单的逻辑和移位操作从 N'_a 中提取。数据结构可以轻松地在 N 之外维护 N'_0 和 N'_1 。

我们令 $N_k = \sum_{j=0}^{k-1} n_j B^j$ (对于 $0 \leq k < r$)。(我们有 $N_r = N$ 且 $N_0 = 0$ 。) 注意 N_k 是大小至多为 B^{k-1} 的块中项的总数，因此 $N - N_k$ 是存储在大小至少为 B^k 的块中的项数。回顾较大的块包含索引较低的项。因此，第 i 个项位于大小至多为 B^k 的块中，当且仅当 $N - N_{k+1} \leq i$ 。(回顾索引从 0 开始。)

数 N_k ($0 \leq k < r$) 可以由数据结构显式维护，而不影响各种操作的渐近运行时间。或者， $N_k = (N'_0 \wedge M_k) + B(N'_1 \wedge M_k)$ ，其中 $\wedge$ 是按位逻辑与操作， M_k 是 kb 个 1 的掩码，即 $M_k = 2^{kb} - 1 = (1 \ll kb) - 1$ ，其中 $\ll$ 表示左移。(掩码可以预先计算。) 乘以 B 也可以替换为左移 b 位，即 $N_k = (N'_0 \wedge M_k) + (N'_1 \wedge M_k) \ll b$ 。

为了访问可调整大小数组中的第 i 个项 ($0 \leq i < N$)，我们需要找到唯一的 $0 \leq k < r$ ，使得 $N - N_{k+1} \leq i < N - N_k$ 。第 i 个项然后在大小为 B^k 的第 $\left\lfloor \frac{i - (N - N_{k+1})}{B^k} \right\rfloor$ 个块的第 $(i - (N - N_{k+1})) \bmod B^k$ 个位置。(如果 $k = 0$ ，则第 i 个项是大小为 B 的单个部分填充块中的第 $(i - (N - N_1))$ 个项。) 由于 B^k 是 2 的幂，一旦知道 k ，第 i 个项就可以使用逻辑和移位操作在常数时间内轻松访问。(注意 $x \bmod B^k = x \wedge M_k$ 且 $x/B^k = x \gg bk$ ，其中 $\gg$ 表示右移。)

令 $x = (N - 1) - i$ 为第 i 个项从数组末端算起的索引，再次从 0 开始编号。我们然后需要找到唯一的 $0 \leq k < r$ ，使得 $N_k \leq x < N_{k+1}$ 。

令 $x = (x_{r-1}, \dots, x_1, x_0)_B$ 为 x 的标准 B 进制表示。为找到 k ，我们首先找到最大的索引 ℓ 使得 $x_\ell > 0$ 。这可以通过找到 x 中最高有效设置位的索引轻松完成，即

$\ell \leftarrow \text{msb}(x)/b$ 。(我们再次假设索引从 0 开始。)

由于 $x > 0$, 我们有 $x \geq B^\ell$ 。又 $N_{\ell+1} \leq 2B \sum_{j=0}^{\ell-1} B^j = 2B \frac{B^\ell - 1}{B - 1} \leq 3B^\ell$, 假设 $B \geq 4$ 。因此, $N_{\ell+1} > x$, 故 $k \leq \ell - 1$ 。如果 $x < N_\ell$, 则 $k = \ell - 1$ 。否则, 如果 $N_\ell \leq x < N_{\ell+1}$, 则当然 $k = \ell$ 。如果 $N_{\ell+1} \leq x$, 则 $k = \ell'$, 其中 $\ell' > \ell$ 是使得 $n_{\ell'} > 0$ 的最小索引。(这样的索引必定存在, 因为 $x < N$ 。)这等价于找到最小的 $\ell' > \ell$, 使得 $n_{\ell'}^0 > 0$ 或 $n_{\ell'}^1 > 0$ 。这可以如下轻松完成:

$$\ell' \leftarrow \text{lsb}((N'_0 \vee N'_1) \wedge (\neg M_{\ell+1}))/b,$$

其中 $\text{lsb}(x)$ 是 x 中最低有效设置位的索引, $\vee$ 是逻辑或操作, $\neg$ 表示取补。

如果 x 是一个数, 则 $x \wedge (-x)$ 仅保留 x 中最右边的 1。(参见 Knuth [12] 中的方程 (37)。) 因此, lsb 可以容易地归约到 msb , 即 $\text{lsb}(x) = \text{msb}(x \wedge (-x))$ 。(相反的归约是未知的。)

计算一个数的最重要设置位的索引等价于计算二进制对数, 即 $\text{msb}(x) = \lfloor \lg x \rfloor$ 。Knuth [12] 讨论了在常数时间内找到一个字中最高有效设置位的索引的几种方法。一种选择(如果可用)是将 x 转换为浮点数并提取指数。Fredman 和 Willard [7] 描述了一种巧妙的方法, 使用常数次数的逻辑、移位和算术操作来计算 $\text{msb}(x)$ 。他们的方法需要使用乘法。乘法的使用是**本质的**。Brodal (参见 Knuth [12]) 设计了一种使用标准逻辑、算术和移位操作计算 $\text{msb}(x)$ 的方法, 不使用乘法, 需要 $O(\log \log w)$ 时间, 其中 w 是机器字中的位数。Brodnik 等人 [3] 获得了匹配的下界。

获得独立于 r 的常数访问时间仅具有理论意义, 因为在实践中使用的 r 值预期非常小。简单的 $O(\log r)$ 二分查找算法, 甚至朴素的 $O(r)$ 算法(用于定位 k 的值), 可能效果最佳。

7 7 数据结构变换

如果 r 是常数, 则前一节的 $(O(rN^{1/r}), O(N^{1-1/r}))$ -实现是最优的, 无论是在使用的临时存储量上, 还是在其 $O(r) = O(1)$ 的增长和收缩操作摊销成本上。如果 $r = r(N)$ 是 N 的一个(缓慢)增长的函数, 有趣的问题是存储数组所需的 $O(rN^{1/r})$ 额外空间能否减少到 $O(N^{1/r})$, 以及增长和收缩操作的摊销成本是否必须是 $\Omega(r)$ 。我们可以假设 $r(N) = O(\log N)$, 因为 $N^{1/O(\log N)} = O(1)$ 。

我们在此证明, 额外空间量可以降低到 $O(N^{1/r})$, 至少如果 $r(N) \leq \frac{1}{2} \frac{\log N}{\log \log N}$ 。在第 9 节中, 我们证明增长和收缩操作的摊销成本必须是 $\Omega(r)$ 。

我们使用一个令人惊讶的简单变换将所需的额外空间从 $O(rN^{1/r})$ 降低到 $O(N^{1/r})$ 。为陈述该变换, 我们需要以下定义:

定义 7.1 (标准 $(s(N), t(N))$ -实现)。一个 $(s(N), t(N))$ -实现是**标准的**, 如果它满足以下条件: (i) 当分配一个新块 B 时, 来自其他块 $A_1, A_2, \dots, A_k$ 的项立即按顺序复制到其中。更具体地说, 首先 A_1 的所有项被复制到 B 的前几个位置, 然后 A_1 被释放,

然后 A_2 的项被复制到 B 的下几个位置, A_2 被释放, 以此类推。(ii) 在每次这样的操作之后, 数据结构使用的总空间为 $N + s(N)$ 。

换句话说, 一个 $(s(N), t(N))$ -实现是标准的, 如果 $t(N)$ 的临时额外空间仅在新块分配之后的短暂时间内需要。不难检验本文中给出的所有实现都是标准的。我们现在可以获得以下结果。

引理 7.2. 标准的 $(s(N), O(N))$ -实现可以转换为 $(s(N) + O(\frac{N}{t(N)}), s(N) + O(t(N)))$ -实现 (对于任意非递减函数 $t(N)$), 而不增加访问操作的渐近最坏情况成本以及增长和收缩操作的渐近摊销成本。

证明。 鉴于标准实现的定义, 我们只需要考虑一个要接收当前存储在 $A_1, A_2, \dots, A_k$ 中的项的新块 B 的分配。令 b 为 B 的长度。如果 $b \leq t(N)$, 没有问题。否则, 我们不分配 B , 而是顺序分配并填充 $b/t(N)$ 个大小为 $t(N)$ 的块 $B_1, B_2, \dots, B_{b/t(N)}$ 。我们当然需要维护指向这些块的指针。由于在任何给定时间分配的所有块的总大小最多为 $N + s(N)$, 所需的额外指针数最多为 $(N + s(N))/t(N)$ 。存储大小为 N 的数组所需的总空间最多为 $N + s(N) + O(\frac{N}{t(N)})$ 。数据结构在任何时间所需的总空间最多为 $N + s(N) + O(\frac{N}{t(N)}) + t(N)$, 即 $s(N) + O(t(N))$, 因为我们可以假设 $t(N) \leq N$ 。

利用该引理, 我们轻松得到以下定理:

定理 7.3. 对于任意整数 $r = r(N) \leq \frac{1}{2} \frac{\log N}{\log \log N}$, 存在一个 $(O(N^{1/r}), O(N^{1-1/r}))$ -实现, 在 $O(1)$ 最坏情况时间内支持访问和修改操作, 并在 $O(r)$ 摊销时间内支持增长和收缩操作。

证明。 将引理 7.2 的变换 (取 $t(N) = N^{1-1/r}$) 应用于第 6 节的标准 $(O(rN^{1/(2r)}), O(N^{1-1/(2r)}))$ -实现。结果是一个 $(O(rN^{1/(2r)} + N^{1/r}), O(N^{1-1/r}))$ -实现。如果 $r \leq \frac{1}{2} \frac{\log N}{\log \log N}$, 则 $2r \log r \leq \log N$, 因此 $rN^{1/(2r)} \leq N^{1/r}$ 。从而获得的实现满足定理的要求。

8 8 增长游戏

为了获得关于真正最优数据结构应如何处理增长操作的指示, 以及为了获得标准数据结构增长操作摊销成本的 $\Omega(r)$ 下界 (该数据结构仅被允许 $N + O(N^{1/r})$ 空间来存储大小为 N 的数组), 我们定义并分析一个有趣的单人游戏, 称为**增长游戏**, 它是标准数据结构处理增长操作方式的抽象。(见定义 7.1。)

8.1 8.1 (N, k, Δ) -增长游戏

令 N, k 和 Δ 为固定参数。我们需要将 N 个项一个接一个地插入 k 个初始为空的子数组 $A_1, A_2, \dots, A_k$ 中, 这些子数组代表一个可调整大小数组 A 。 A 的前几个项存储在 A_k 中, 接下来的存储在 A_{k-1} 中, 依此类推。(这种表面上的反转实际上简化了事情。) 令 $a_1, a_2, \dots, a_k$ 分别为 $A_1, A_2, \dots, A_k$ 中的项数。

一些子数组 $A_1, A_2, \dots, A_k$ 可能为空, 即不包含项。(如果 A_i 为空, 则 $a_i = 0$ 。) 空子数组 (如果有的话) 假定为前几个子数组, 即如果 $a_i = 0$, 则 $a_1 = a_2 = \dots = a_i = 0$ 。第一个非空子数组最多可包含 Δ 个空位。所有其他子数组都是满的, 即为它们分配的内存块的大小等于存储在其中的项数。存储这些项使用的额外空间因此总是最多为 $k + \Delta$ 。

增长操作 (向数组添加新项) 的实现如下。如果第一个非空子数组有空位, 该项简单地被放在其第一个空位中。(见图 6(a)。) 如果所有非空子数组都是满的, 但至少有一个空子数组, 则分配一个大小的新子数组 $\Delta + 1$, 新项被放在其第一个位置。剩余的 Δ 个位置留空。此子数组替换最后一个空子数组。(见图 6(b)。) 在这两种情况下, 插入新项的成本定义为 1。(为简单起见, 我们忽略分配成本。) 注意在前两种情况下, 游戏的玩家没有真正的选择。

当所有 k 个子数组都是满的时候, 游戏变得更有意思。我们现在有 k 种可能的移动。对于每个 $i \in [k] = \{1, 2, \dots, k\}$, 我们可以分配一个大小的新子数组 $(\Delta + 1) + \sum_{j=1}^i a_j$, 将 $A_i, A_{i-1}, \dots, A_1$ 中的项按此顺序复制到新子数组中, 然后将新项放在新子数组的第一个空位。(新子数组仍有 Δ 个空位。) 旧子数组 $A_1, \dots, A_i$ 被释放。新子数组成为 A_i , 而 $A_1, A_2, \dots, A_{i-1}$ 变为空。(见图 6(c)。) 操作成本定义为 $1 + \sum_{j=1}^i a_j$ 。(再次, 我们只计算项赋值, 忽略分配和释放的成本。) 第二种类型的移动实际上是第三种类型的移动, 但 i 被强制为使得 $a_i = 0$ 的最大索引。

定义 8.1 (总成本和摊销成本)。 令 $C_{N,k,\Delta}$ 为玩 (N, k, Δ) -增长游戏所需的最小总成本。令 $A_{N,k,\Delta} = C_{N,k,\Delta}/N$ 为单次增长操作相应的摊销成本。

我们对寻找 $C_{N,k,\Delta}$ (以及 $A_{N,k,\Delta}$) 和一个达到这些值的最优移动序列感兴趣。这些问题的解答结果相当简单, 也相当有趣。

$A_{N,cN^{1/r},cN^{1/r}}$ (对于某个常数 $c > 0$) 的值可以用来从下方界出当只允许 $cN^{1/r}$ 额外空间时增长操作的摊销成本, 如我们将看到的。我们在这里更宽松, 因为我们假设 N 是预先知道的, 并且我们即使当数组包含远少于 N 个项时也允许 $cN^{1/r}$ 额外空间。我们也不对增长操作期间临时使用的空间量施加任何限制。这一切只使我们的下界更强。

8.2 8.2 归约到 $\Delta = 0$ 情况

看起来允许一个子数组中最多 Δ 个空位是游戏的一个重要特征, 有助于降低增长操作的摊销成本。我们首先证明 (也许令人惊讶地), 空位帮助不大, 至少对于大多数参数值来说。更具体地说, 我们描述了从 $\Delta > 0$ 情况到 $\Delta = 0$ 情况的一个归约。

引理 8.2。 对于每个能被 $\Delta + 1$ 整除的 N , $C_{N,k,\Delta} = (\Delta + 1)C_{\frac{N}{\Delta+1},k,0}$ 且 $A_{N,k,\Delta} = A_{\frac{N}{\Delta+1},k,0}$ 。

证明。 玩 (N, k, Δ) -增长游戏等价于在 $\Delta + 1$ 个项的块上玩 $(\frac{N}{\Delta+1}, k, 0)$ -增长游戏。为看到这一点, 注意当 (N, k, Δ) -增长游戏中的增长操作留下 Δ 个空位时, 接下来的 Δ 次增长操作必须立即填充这些空位。

8.3 $\Delta = 0$ 情况的分析

鉴于第 8.2 节的简单归约, 只需考虑 $\Delta = 0$ 情况。为简化记号, 令 $C_{N,k} = C_{N,k,0}$ 且 $A_{N,k} = A_{N,k,0}$ 。我们将相应的游戏称为 (N, k) -增长游戏。令 $\mathbb{N} = \{0, 1, \dots\}$ 为非负整数集合。

定义 8.3 (状态及其成本)。 令

$$P_{N,k} = \left\{ a = (a_1, a_2, \dots, a_k) \in \mathbb{N}^k \mid \sum_{i=1}^k a_i = N \text{ 且 } a_i = 0 \Rightarrow a_{i-1} = 0, i = 2, \dots, k \right\},$$

为 (N, k) -增长游戏中所有总大小为 N 的状态的集合。对于 $a \in P_{N,k}$, 令 $C(a) = C_k(a)$ 为从状态 $(0, 0, \dots, 0)$ 出发到达状态 $a = (a_1, a_2, \dots, a_k)$ (即 $|A_i| = a_i$, 对于 $i \in [k]$) 所需的最小总成本。显然 $C_{N,k} = \min\{C(a) \mid a \in P_{N,k}\}$ 。

以下引理给出了计算状态成本的简单递推关系。这些关系本身并不提供计算每个 $a \in P_{N,k}$ 的 $C_{N,k}$ 和 $C(a)$ 的高效方法。然而, 它们使我们能够获得这些量的显式公式。

引理 8.4。 对于每个 $a = (a_1, a_2, \dots, a_k) \in P_{N,k}$:

$$(i) \ C_k(a_1, a_2, \dots, a_k) = C_k(0, \dots, 0, a_k) + C_{k-1}(a_1, a_2, \dots, a_{k-1}).$$

$$(ii) \ C_k(a_1, a_2, \dots, a_k) = \sum_{j=1}^k C_j(0, \dots, 0, a_j).$$

$$(iii) \ C_k(0, \dots, 0, a_k) = C_{a_k-1,k} + a_k.$$

$$(iv) \ C(a) = N + \sum_{j=i}^k C_{a_j-1,j}, \text{ 如果 } 0 = a_{i-1} < a_i.$$

证明。 令 $(a_1, a_2, \dots, a_k) \in P_{N,k}$ 。考虑从 $(0, 0, \dots, 0)$ 到 $(a_1, a_2, \dots, a_k)$ 的移动序列。最后一次改变 A_k 的移动必须到达状态 $(0, \dots, 0, a_k)$ 。到达该状态的最小成本为 $C_k(0, \dots, 0, a_k)$ 。从状态 $(0, \dots, 0, a_k)$ 移动到 $(a_1, a_2, \dots, a_k)$ 的最小成本为 $C_{k-1}(a_1, a_2, \dots, a_{k-1})$, 因为仅能使用 $A_1, A_2, \dots, A_{k-1}$ 。这建立了 (i); (ii) 由归纳得到。

移动到 $(0, \dots, 0, a_k)$ 必须来自状态 $(b_1, b_2, \dots, b_k)$, 其中 $\sum_{j=1}^k b_j = a_k - 1$ 。到达该状态的最小成本恰好为 $C_{a_k-1,k}$, 且移动到 $(0, \dots, 0, a_k)$ 的成本为 a_k 。这建立了 (iii); (iv) 由归纳得到。

我们得到的 $C_{N,k}$ 的精确公式涉及二项式系数。因此, 方便的做法是先陈述两个众所周知的二项式系数恒等式。

引理 8.5。 对于每个 $n, k \geq 0$: (i) $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ 且 (ii) $\sum_{i=0}^k \binom{n+i-1}{i} = \binom{n+k}{k}$ 。

证明。 (i) 是众所周知的。(它甚至可以与 $\binom{0}{0} = 1$ 一起用作二项式系数的定义。注意对于每个 n , $\binom{n}{0} = 1$ 。) (ii) 通过对 k 的简单归纳得到。对于 $k = 0$, 两项均为 1。归纳步骤轻松可得:

$$\sum_{i=0}^k \binom{n+i-1}{i} = \sum_{i=0}^{k-1} \binom{n+i-1}{i} + \binom{n+k-1}{k} = \binom{n+k-1}{k-1} + \binom{n+k-1}{k} = \binom{n+k}{k}.$$

以下定理给出了计算总成本 $C_{N,k}$ 的精确公式, 并刻画了 $P_{N,k}$ 中达到该最小总成本的所有状态。(相应的最优移动序列在第 8.4 节中考虑。)

定理 8.6. (i) 如果 $\binom{n+k-1}{k} - 1 \leq N \leq \binom{n+k}{k} - 1$ (对于某个 $n \geq 0$), 则

$$C_{N,k} = (N+1)n - \binom{n+k}{k+1}.$$

(ii) 如果 $\binom{n+k-1}{k} \leq N < \binom{n+k}{k}$ (对于某个 $n \geq 1$), 则

$$C_{N,k} - C_{N-1,k} = n.$$

(iii) 如果 $\binom{n+k-1}{k} - 1 \leq N \leq \binom{n+k}{k} - 1$ (对于某个 $n \geq 0$) 且 $a \in P_{N,k}$, 则

$$C_{N,k} = C(a) \iff \binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}, \text{ 对于每个 } i \in [k].$$

论断 (i) 中的两个不等式都是弱不等式。因此, 如果 $N = \binom{n+k}{k} - 1$ (对于某个 $n \geq 0$), 则条件 (i) 被 n 和 $n+1$ 同时满足。这是一致的, 因为对于这个 N 的值,

$$\begin{aligned} (N+1)(n+1) - \binom{n+k+1}{k+1} &= (N+1)n + \left(N+1 - \binom{n+k+1}{k+1} \right) \\ &= (N+1)n + \left(\binom{n+k}{k+1} - \binom{n+k+1}{k+1} \right) = (N+1)n - \binom{n+k}{k+1}, \end{aligned}$$

其中最后一个等号由引理 8.5(i) 得到。

定理 8.6(iii) 可以使用以下定义更简洁地陈述:

定义 8.7 ($Q_{N,k}$)。如果 $\binom{n+k-1}{k} \leq N < \binom{n+k}{k}$, 令

$$Q_{N,k} = \left\{ a = (a_1, a_2, \dots, a_k) \in P_{N,k} \mid \binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}, \text{ 对于每个 } i \in [k] \right\}.$$

定理 8.6(iii) 说如果 $a \in P_{N,k}$, 则 $C_{N,k} = C(a)$ 当且仅当 $a \in Q_{N,k}$ 。也就是说, $Q_{N,k}$ 是 (N, k) -增长游戏中**最优最终位置**的集合。对于大多数 N 的值, 存在许多最优最终位置。如果 $N = \binom{n+k}{k} - 1$, 则 $((\binom{n}{1}), (\binom{n+1}{2}), \dots, (\binom{n+k-1}{k}))$ 是唯一的最优最终状态, 总成本为 $\binom{n+k}{k}n - \binom{n+k}{k-1} = \frac{kn}{k+1}\binom{n+k}{k}$ 。(我们在第 8.4 节中对此有更多说明。)

我们通过对 N 和 k 归纳证明定理 8.6。我们称若 $N' < N$ 且 $k' \leq k$, 则 $(N', k') < (N, k)$ 。我们称若 $N' \leq N$ 且 $k' \leq k$, 则 $(N', k') \preceq (N, k)$ 。我们将证明拆分成小块, 每块陈述为一个单独的断言。归纳基础由以下断言建立:

断言 8.8. 定理 8.6 对 $n = 1$ 成立, 即当 $0 \leq N \leq k$ 时。

证明. 如果 $0 \leq N \leq k$, 则 $C_{N,k} = N$, 即每个项的插入成本恰好为 1, 且如果 $a \in P_{N,k}$, 则 $C(a) = N$ 当且仅当 $0 \leq a_i \leq 1$ (对于每个 $i \in [k]$)。

在建立归纳步骤之前, 我们证明在某种意义下, 定理 8.6(iii) 蕴含定理 8.6(i), 且定理 8.6(i) 蕴含定理 8.6(ii)。

断言 8.9. 如果定理 8.6(i) 对 (N, k) 和 $(N-1, k)$ 成立, 则定理 8.6(ii) 对 (N, k) 成立, 即如果 $\binom{n+k-1}{k} \leq N < \binom{n+k}{k}$ (对于某个 $n \geq 1$), 则 $C_{N,k} - C_{N-1,k} = n$ 。

证明。 如果 $\binom{n+k-1}{k} \leq N < \binom{n+k}{k}$ (对于某个 $n \geq 1$)，则

$$\binom{n+k-1}{k} - 1 < N \leq \binom{n+k}{k} - 1, \quad \binom{n+k-1}{k} - 1 \leq N - 1 < \binom{n+k}{k} - 1.$$

定理 8.6(i) 应用于 (N, k) 和 $(N-1, k)$ 给出 $C_{N,k} - C_{N-1,k} = n$ 。(注意这里我们使用了定理 8.6(i) 只需要弱不等式 $N \leq \binom{n+k}{k} - 1$ 这一事实。)

断言 8.10。 如果定理 8.6(i) 对每个 $(N', k') < (N, k)$ 成立， $\binom{n+k-1}{k} - 1 \leq N < \binom{n+k}{k} - 1$ (对于某个 $n \geq 1$)，且 $a \in Q_{N,k}$ (即 $a \in P_{N,k}$ 且对于每个 $i \in [k]$ 有 $\binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}$)，则

$$C(a) = (N+1)n - \binom{n+k}{k+1}.$$

(注意这是声称的 $C_{N,k}$ 的值。)

证明。 假设 $a \in P_{N,k}$ 且对于每个 $i \in [k]$ 有 $\binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}$ 。则

$$\begin{aligned} C(a) &= N + \sum_{i=1}^k C_{a_i-1, i} \\ &= N + \sum_{i=1}^k \left(a_i(n-1) - \binom{n+i-1}{i+1} \right) \\ &= (N+1)n - \left(1 + (n-1) + \sum_{i=1}^k \binom{n+i-1}{i+1} \right) \\ &= (N+1)n - \sum_{j=0}^{k+1} \binom{n+j-2}{j} = (N+1)n - \binom{n+k}{k+1}. \end{aligned}$$

在第一行我们使用了引理 8.4，利用了对于每个 $i \in [k]$ 有 $a_i \geq 1$ 这一事实。在第二行我们对 $(a_i - 1, i) < (N, k)$ 使用了定理 8.6(i)。第三行由简单重排得到。第四行由引理 8.5 得到。注意

$$\sum_{j=0}^{k+1} \binom{n+j-2}{j} = \sum_{j=0}^{k+1} \binom{(n-1)+j-1}{j} = \binom{(n-1)+(k+1)}{k+1} = \binom{n+k}{k+1}.$$

断言 8.11。 假设 $\binom{n+k-1}{k} - 1 \leq N \leq \binom{n+k}{k} - 1$ (对于某个 $n \geq 1$)，且 $a \in P_{N,k}$ 。

(i) 如果对于某个 $i \in [k]$ 有 $a_i < \binom{n+i-2}{i}$ ，则存在 $j \in [k]$ 使得 $a_j > \binom{n+j-2}{j}$ 。

(ii) 如果对于某个 $j \in [k]$ 有 $a_j > \binom{n+j-1}{j}$ ，则存在 $i \in [k]$ 使得 $a_i < \binom{n+i-1}{i}$ 。

证明。 (i) 如果对于每个 $j \in [k]$ 有 $a_j \leq \binom{n+j-2}{j}$ ，则

$$N = \sum_{j=1}^k a_j < \sum_{j=1}^k \binom{n+j-2}{j} = \binom{n+k-1}{k} - 1,$$

与 n 的定义矛盾。

(ii) 类似地, 如果对于每个 $i \in [k]$ 有 $a_i \geq \binom{n+i-1}{i}$, 则

$$N = \sum_{i=1}^k a_i > \sum_{i=1}^k \binom{n+i-1}{i} = \binom{n+k}{k} - 1,$$

再次与 n 的定义矛盾。

断言 8.12. 如果定理 8.6(ii) 对每个 $(N', k') < (N, k)$ 成立, $\binom{n+k-1}{k} - 1 \leq N < \binom{n+k}{k} - 1$ (对于某个 $n \geq 1$), 且 $a \in P_{N,k} \setminus Q_{N,k}$ (即 $a \in P_{N,k}$ 但对于某个 $i \in [k]$, $\binom{n+i-2}{i} \leq a_i \leq \binom{n+i-1}{i}$ 不被满足), 则存在 $b \in P_{N,k}$ 使得 $C(a) > C(b)$ 。作为推论, $C(a) > C_{N,k}$ 。

证明. 有两种 (非互斥的) 情况:

情况 1: 存在 $i \in [k]$ 使得 $a_i < \binom{n+i-2}{i}$ 。

由断言 8.11(i), 存在 $j \in [k]$ 使得 $a_j > \binom{n+j-2}{j}$ 。令 $b = (b_1, b_2, \dots, b_k) \in P_{N,k}$ 满足 $b_i = a_i + 1$, $b_j = a_j - 1$, 且对于 $\ell \in [k] \setminus \{i, j\}$ 有 $b_\ell = a_\ell$ 。由于假设定理 8.6(ii) 对 (a_i, i) 和 $(a_j - 1, j)$ 成立, 我们有 $C_{a_i, i} - C_{a_i-1, i} \leq n - 2$ 而 $C_{a_j-1, j} - C_{a_j-2, j} \geq n - 1$ 。因此,

$$C(b) - C(a) = (C_{a_i, i} - C_{a_i-1, i}) - (C_{a_j-1, j} - C_{a_j-2, j}) < 0.$$

情况 2: 存在 $j \in [k]$ 使得 $a_j > \binom{n+j-1}{j}$ 。

由断言 8.11(ii), 存在 $i \in [k]$ 使得 $a_i < \binom{n+i-1}{i}$ 。再次令 $b = (b_1, b_2, \dots, b_k) \in P_{N,k}$ 满足 $b_i = a_i + 1$, $b_j = a_j - 1$, 且对于 $\ell \in [k] \setminus \{i, j\}$ 有 $b_\ell = a_\ell$ 。由于假设定理 8.6(ii) 对 (a_i, i) 和 $(a_j - 1, j)$ 成立, 我们有 $C_{a_i, i} - C_{a_i-1, i} \leq n - 1$ 而 $C_{a_j-1, j} - C_{a_j-2, j} \geq n$ 。因此,

$$C(b) - C(a) = (C_{a_i, i} - C_{a_i-1, i}) - (C_{a_j-1, j} - C_{a_j-2, j}) < 0.$$

我们现在有了归纳证明定理 8.6 所需的所有部分。

定理 8.6 的证明. 归纳基础由断言 8.8 建立, 该断言表明定理对 $0 \leq N \leq k$ 成立。因此我们需要证明如果定理对每个 $(N', k') < (N, k)$ (对于某个 $N > k$) 成立, 则它对 (N, k) 也成立。注意如果 $N > k$ 且 $N \leq \binom{n+k}{k}$, 则 $n \geq 2$ 。断言 8.10 表明对于每个 $a \in Q_{N,k}$, 我们有 $C(a) = (N+1)n - \binom{n+k}{k+1}$ 。断言 8.12 表明如果 $a \in P_{N,k} \setminus Q_{N,k}$, 则 $C(a) > C_{N,k}$ 。由此得到 $C_{N,k} = (N+1)n - \binom{n+k}{k+1}$, 建立了对 (N, k) 的定理 8.6(i) 和 (iii)。断言 8.9 然后建立了定理 8.6(ii)。这完成了归纳步骤的证明, 从而完成了定理 8.6 的证明。

定理 8.6 的一个方便推论如下:

推论 8.13. (i) 如果 $N = \binom{n+k}{k} - 1$, 则 $C_{N,k} = \frac{kn}{k+1}(N+1)$ 。

(ii) 如果 $\binom{n+k-1}{k} \leq N \leq \binom{n+k}{k} - 1$, 则 $C_{N,k} \geq \frac{k(n-1)}{k+1}N$, 因此 $A_{N,k} \geq \frac{k(n-1)}{k+1} \geq \frac{1}{2}(n-1)$ 。

证明. 为证 (i), 注意如果 $N = \binom{n+k}{k} - 1$, 则

$$C_{N,k} = \binom{n+k}{k}n - \binom{n+k}{k+1} = \binom{n+k}{k}n - \frac{n}{k+1}\binom{n+k}{k} = \frac{kn}{k+1}\binom{n+k}{k} = \frac{kn}{k+1}(N+1).$$

对于每个 $n \geq 1$, 我们对 N 归纳证明 (ii)。如果 $N = \binom{n+k-1}{k}$, 则 $C_{N,k} > C_{N-1,k} = \frac{k(n-1)}{k+1}N$ 。现在, 如果 $\binom{n+k-1}{k} < N < \binom{n+k}{k}$, 则由定理 8.6(ii),

$$C_{N,k} = C_{N-1,k} + n \geq \frac{k(n-1)}{k+1}(N-1) + n \geq \frac{k(n-1)}{k+1}N.$$

8.4 8.4 最优玩法——二项式计数器

(N, k) -增长游戏的最优解 (即最小总成本的移动序列) 可以借助定理 8.6 和引理 8.4 轻松重建。一般来说, 这样的序列不是唯一的。最优序列仅在 $N = \binom{n+k}{k} - 1$ (对某个 $n \geq 0$) 时是唯一的。

引理 8.14. (N, k) -增长游戏有唯一的最优序列当且仅当 $N = \binom{n+k}{k} - 1$ (对于某个 $n \geq 0$)。

证明。 回顾 $Q_{N,k}$ 是最优最终状态的集合。我们首先证明 $|Q_{N,k}| = 1$ 当且仅当 $N = \binom{n+k}{k} - 1$ (对于某个 $n \geq 0$)。这很容易得到, 因为 $\sum_{i=1}^k \binom{n-i+1}{i} = \binom{n+k}{k} - 1 = N$ 。因此, 如果 $N = \binom{n+k}{k} - 1$, 则唯一的最优最终状态是 $a_i = \binom{n-i+1}{i}$ (对于 $i \in [k]$)。如果 N 不具有此形式, 则 $|Q_{N,k}| > 1$, 因为在选择各个 a_i 时有一定的自由度。最后, 如果 $N = \binom{n+k}{k} - 1$, 则由引理 8.4, 所有中间状态也具有此形式, 最优序列是唯一的。

$N = \binom{n+k}{k} - 1$ 的唯一最优移动序列可以使用二项式计数器来表达, 其伪代码如图 7 所示。除了子数组的大小 $(a_1, a_2, \dots, a_k)$, 我们还维护一个计数器 $(b_1, b_2, \dots, b_k)$ 。Initialize(k) 将 $a_1, a_2, \dots, a_k$ 和 $b_1, b_2, \dots, b_k$ 设为 0。它还将 b_{k+1} 设为 ∞ 。Increment(k) 递增计数器并执行 (N, k) -增长游戏中的一次移动, 如下: 它找到最小的 i 使得 $b_i < b_{i+1}$ 。(如我们将看到的, 这将满足 $b_1 = b_2 = \dots = b_i < b_{i+1}$ 。)然后执行游戏中的第 i 次移动, 即 $a_i \leftarrow 1 + \sum_{j=1}^i a_j$, 而 $a_1, a_2, \dots, a_{i-1} \leftarrow 0$ 。(这对应于合并 $A_1, A_2, \dots, A_i$ 。)然后它通过令 $b_i \leftarrow b_i + 1$ 且 $b_1, b_2, \dots, b_{i-1} \leftarrow 0$ 来递增计数器。二项式计数器在行动中的一个例子 ($k = 4$) 如图 8 所示。

Initialize(k): $a_1, a_2, \dots, a_k \leftarrow 0$ $b_1, b_2, \dots, b_k \leftarrow 0$ $b_{k+1} \leftarrow \infty$	Increment(k): $i \leftarrow \min\{j \in [k] \mid b_j < b_{j+1}\}$ $a_i \leftarrow 1 + \sum_{j=1}^i a_j$ $b_i \leftarrow b_i + 1$ $a_1, a_2, \dots, a_{i-1} \leftarrow 0$ $b_1, b_2, \dots, b_{i-1} \leftarrow 0$
---	--

图 7: 二项式计数器的伪代码。

引理 8.15. 在任何时候, 对于每个 $i \in [k]$, 有 $a_i = \binom{b_i+i-1}{i}$ 。

证明。对计数器的步数归纳。初始条件成立，因为对于 $i \in [k]$, $\binom{i-1}{i} = 0$ 。假设断言对当前的 $(a_1, a_2, \dots, a_k)$ 和 $(b_1, b_2, \dots, b_k)$ 成立，且 $b_1 = b_2 = \dots = b_i = n < b_{i+1}$ 。令 $a_i = 1 + \sum_{j=1}^i a_j$ 和 $b_i = b_i + 1$ 为 a_i 和 b_i 的新值。由归纳假设和引理 8.5(ii)，我们得到

$$a_i = 1 + \sum_{j=1}^i a_j = 1 + \sum_{j=1}^i \binom{n+j-1}{j} = \sum_{j=0}^i \binom{n+j-1}{j} = \binom{n+i}{i} = \binom{b_i+i-1}{i}.$$

由此我们得到：

引理 8.16。如果对于某个 $n \geq 0$ 有 $b_1 = b_2 = \dots = b_k = n$ ，则 $N = \sum_{i=1}^k a_i = \binom{n+k-1}{k} - 1$ ，且二项式计数器的移动序列是 (N, k) -增长游戏的唯一最优解。

8.5 8.5 扩展规则

在本节中，我们证明 (N, k) -增长游戏的规则可以稍作扩展而不改变最优值。游戏的第三条规则(如图 6(c) 所示)可以写为 $(a_1, a_2, \dots, a_k) \rightarrow (0, \dots, 0, 1 + \sum_{j=1}^i a_j, a_{j+1}, \dots, a_k)$ (对于某个 $i \in [k]$)。我们首先允许移动

$$(a_1, a_2, \dots, a_k) \rightarrow (0, \dots, 0, a_1, \dots, a_{i-1}, \sum_{\ell=i}^j a_\ell + 1, \dots, a_k),$$

对于 $1 \leq i < j \leq k$ ，其中 $A_i, A_{i+1}, \dots, A_j$ 被合并为一个新的 A_j ，子数组 $A_1, \dots, A_{i-1}$ 变为 $A_{j-i+1}, \dots, A_{j-1}$ ，且 $A_1, \dots, A_{j-i}$ 变为空。(此规则不添加新项。)本文中描述的许多算法使用此类移动。移动的成本为 $\sum_{\ell=i}^j a_\ell$ 。注意子数组中项的顺序仍被保持。

引理 8.17。允许移动 $(a_1, a_2, \dots, a_k) \rightarrow (0, \dots, 0, a_1, a_2, \dots, a_{i-1}, \sum_{\ell=i}^j a_\ell, a_{j+1}, \dots, a_k)$ (对于 $1 \leq i < j \leq k$) 不改变 (N, k) -增长游戏的最优值。

证明。我们证明对于每个 $a \in P_{N,k}$ 和每个 $1 \leq i < j \leq k$ ，有

$$C_k(0, \dots, 0, a_1, a_2, \dots, a_{i-1}, \sum_{\ell=i}^j a_\ell, a_{j+1}, \dots, a_k) \leq C_k(a_1, a_2, \dots, a_k) + \sum_{\ell=i}^j a_\ell,$$

表明新移动不提供任何优势。由引理 8.4，我们有 $C_k(a_1, a_2, \dots, a_k) = C_j(a_1, a_2, \dots, a_j) + C_k(0, \dots, 0, a_{j+1}, \dots, a_k)$ ，且不等式左边出现的项类似。因此，只需对 $j = k$ 证明该不等

式。现在,

$$\begin{aligned}
& C_k(0, \dots, 0, a_1, a_2, \dots, a_{i-1}, \sum_{\ell=i}^k a_\ell) \\
& \leq C_{k-1}(0, \dots, 0, a_1, a_2, \dots, a_{i-1}) + C_k(0, \dots, 0, \sum_{\ell=i}^k a_\ell) \\
& \leq C_{k-1}(0, \dots, 0, a_1, a_2, \dots, a_{i-1}) + C_k(0, \dots, 0, a_i, a_{i+1}, \dots, a_k - 1) + \sum_{\ell=i}^k a_\ell \\
& \leq C_{k-1}(0, \dots, 0, a_1, a_2, \dots, a_{i-1}) + C_k(0, \dots, 0, a_i, a_{i+1}, \dots, a_k) + \sum_{\ell=i}^k a_\ell \\
& \leq C_k(a_1, a_2, \dots, a_k) + \sum_{\ell=i}^j a_\ell.
\end{aligned}$$

最后一个不等式通过使用引理 8.4(iv) 展开各项得到。注意 $C_k(a_1, a_2, \dots, a_k)$ 展开为 $\sum_{\ell: a_\ell > 0} C_{a_\ell-1, \ell} + \sum_{\ell=1}^k a_\ell$, 而前一行展开为 $\sum_{\ell: a_\ell > 0} C_{a_\ell-1, k'_\ell} + \sum_{\ell=1}^k a_\ell$, 其中对于每个 $\ell \in [k]$ 有 $k'_\ell \leq \ell$ 。更精确地说, 如果 $1 \leq \ell \leq i-1$, 则 $k'_\ell = (k-i) + \ell$; 如果 $i \leq \ell \leq k$, 则 $k'_\ell = \ell$ 。(显然对于 $k' \leq k$ 有 $C_{N, k'} \leq C_{N, k}$ 。)

我们接下来证明允许不保持子数组中项的顺序的一般合并操作也不改变 (N, k) -增长游戏的解。更精确地说, 对于每个 $I = \{i_1, i_2, \dots, i_r\} \subseteq [k]$ (其中 $i_1 < i_2 < \dots < i_r$), 我们允许一个 I -移动合并 $A_{i_1}, A_{i_2}, \dots, A_{i_r}$ 和一个新项。新子数组成为新的 A_{i_r} 。子数组 $A_{i_1}, A_{i_2}, \dots, A_{i_{r-1}}$ 变为空。所有其他子数组不变。换句话说, $a_{i_r} \leftarrow 1 + \sum_{\ell=1}^r a_{i_\ell}$, 而 $a_{i_\ell} \leftarrow 0$ (对于 $\ell \in [r-1]$)。增长游戏中的标准移动是 $[i]$ -移动 (对于 $i \in [k]$)。(注意让 A_{i_r} 成为新合并数组的决定是任意的, 因为 i_r 仅仅是指派给新数组的索引。然而, 这是方便的选择, 因为否则我们需要在状态上定义新的成本函数 $C'(a)$, 等于 $C(\bar{a})$ 的最小值, 其中 $\bar{a}$ 是 a 的一个排列。)我们也可以允许一种不添加新项的 I -移动变体。以下引理的证明本质上与引理 8.17 的证明相同, 因此省略。

引理 8.18. 允许任何 $I \subseteq [k]$ 的 I -移动不改变 (N, k) -增长游戏的最优值。

有了这些扩展, 增长游戏现在捕获了标准实现可以执行的所有移动。(见定义 7.1。)

9 9 增长操作摊销成本的下界

我们现在可以证明对于仅使用 $N + O(rN^{1/r})$ 空间存储大小为 N 的数组的标准实现, 增长操作的摊销成本为 $\Omega(r)$ 的下界, 表明第 6 节的构造是本质上的最优的。

定理 9.1. 任何仅使用 $N + O(rN^{1/r})$ 空间存储大小为 N 的数组的标准可调整大小数组数据结构 (其中 $r = O(\log N)$), 其增长操作的摊销成本必须为 $\Omega(r)$ 。

证明. 假设数据结构使用的额外空间为 $rN^{1/r}$ 。(如果额外空间为 $\gamma rN^{1/r}$ (对于某个常数 $\gamma > 1$), 则对 $r-1$ 应用该定理, 依赖于对于充分大的 N 有不等式 $\gamma rN^{1/r} <$

$(r-1)N^{1/(r-1)}$ 。) 由于任何标准实现的操作对应于（扩展的） $(N, rN^{1/r}, rN^{1/r})$ -增长游戏中的移动，推论 8.13 意味着增长操作的摊销成本至少为

$$A_{N, rN^{1/r}, rN^{1/r}} = A_{\frac{1}{r}N^{1-1/r}, rN^{1/r}} \geq \frac{1}{2}n,$$

只要 $\binom{n+rN^{1/r}}{rN^{1/r}} < \frac{1}{r}N^{1-1/r}$ 。因此，我们想证明当对于某个 $\varepsilon > 0$ 有 $n = \varepsilon r$ 时，对于充分大的 N 有 $\binom{\varepsilon r + rN^{1/r}}{r} < \frac{1}{r}N^{1-1/r}$ 。

为给二项式系数上界，我们使用众所周知的不等式 $\binom{n}{k} \leq \left(\frac{en}{k}\right)^k$ 。现在：

$$\binom{\varepsilon r + rN^{1/r}}{r} \leq \binom{(\varepsilon + 1)rN^{1/r}}{r} \leq \left(\frac{e(\varepsilon + 1)N^{1/r}}{1}\right)^r = (e(\varepsilon + 1))^r N.$$

选择例如 $\varepsilon = \frac{1}{2}$ ，并假设 $3 \leq r \leq \log N$ （对于适当的 δ 选择），我们得到 $r(3e)^{r/2}N^{1/2} \leq N^{3/4} \leq N^{1-1/r}$ ，如所需。

10 10 总结性评论

我们提出了一系列可调整大小数组实现，它们在存储数组所需的空间、调整数组大小（即增长或收缩）所需的空间以及增长和收缩操作的摊销成本之间提供了最优权衡，同时仍保持 $O(1)$ 最坏情况访问成本。我们相信这几乎完全解决了一个非常基本的问题。

我们对增长操作摊销成本的下界仅适用于我们称为**标准**的算法。我们相信这些下界可以扩展到覆盖所有算法，只需假设对项和指针施加适当的不可压缩性假设。（见定理 4.1 证明之后的讨论。）换句话说，我们相信非标准算法相对于标准算法不提供任何优势，而严格证明这一点在概念上应该不难。

为获得增长操作的摊销下界，我们定义了一个有趣的增长游戏并对其进行了精确求解。虽然我们相信游戏的精确解是有启发性的，但知道是否存在一种更简单的方法来获得游戏值的下界（也许通过归纳），足以证明渐近最优的摊销下界，将会很有趣。

11 参考文献

- [1] Philip Bille, Anders Roy Christiansen, Mikko Berggren Ettienne, and Inge Li Gortz. Fast dynamic arrays. In *Proc. of 25th ESA*, volume 87 of LIPIcs, pages 16:1-16:13, 2017.
- [2] Andrej Brodnik, Svante Carlsson, Erik D. Demaine, J. Ian Munro, and Robert Sedgwick. Resizable arrays in optimal time and space. In *Proc. of 6th WADS*, pages 37-48, 1999.
- [3] Andrej Brodnik, Peter Bro Miltersen, and J. Ian Munro. Trans-dichotomous algorithms without multiplication — some upper and lower bounds. In *Proc. of 4th

WADS*, volume 1272 of Lecture Notes in Computer Science, pages 426-439. Springer, 1997.

[4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to algorithms*. MIT press, 3rd edition, 2009.

[5] Paul F. Dietz. Optimal algorithms for list indexing and subset rank. In *Proc. of 1st WADS*, volume 382 of Lecture Notes in Computer Science, pages 39-46. Springer, 1989.

[6] Michael L. Fredman and Michael E. Saks. The cell probe complexity of dynamic data structures. In David S. Johnson, editor, *Proc. of 21st STOC*, pages 345-354. ACM, 1989.

[7] Michael L. Fredman and Dan E. Willard. Surpassing the information theoretic bound with fusion trees. *J. Comput. Syst. Sci.*, 47(3):424-436, 1993.

[8] Michael T. Goodrich and John G. Kloss II. Tiered vectors: Efficient dynamic arrays for rank-based sequences. In *Proc. of 6th WADS*, pages 205-216, 1999.

[9] Stelios Joannou and Rajeev Raman. An empirical evaluation of extendible arrays. In *Proc. of 10th SEA*, volume 6630 of Lecture Notes in Computer Science, pages 447-458. Springer, 2011.

[10] Haim Kaplan, Robert E. Tarjan, Or Zamir, and Uri Zwick. Simulating a stack using queues. In *Proc. of 33rd SODA*, pages 1901-1924, 2022.

[11] Jyrki Katajainen. Worst-case-efficient dynamic arrays in practice. In *Proc. of 15th SEA*, volume 9685 of Lecture Notes in Computer Science, pages 167-183. Springer, 2016.

[12] Donald E Knuth. *The Art of Computer Programming, Volume 4, Fascicle 1: Bitwise Tricks Techniques; Binary Decision Diagrams*. Addison-Wesley Professional, 2009.

[13] J. Ian Munro and S. Srinivasa Rao. Succinct representation of data structures. In *Handbook of Data Structures and Applications*, pages 595-610. Chapman and Hall/CRC, 2018.

[14] Rajeev Raman, Venkatesh Raman, and S. Srinivasa Rao. Succinct dynamic data structures. In *Proc. of 7th WADS*, volume 2125 of Lecture Notes in Computer Science, pages 426-437. Springer, 2001.

[15] Edward Sitariski. Algorithm alley: HATs: Hashed array trees. *Dr. Dobb's Journal of Software Tools*, 21(9):107-??, September 1996.

[16] Wikipedia contributors. Dynamic array — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Dynamic_array&oldid=1078194534, 2022. [Online; accessed 24-March-2022].

[17] Wikipedia contributors. Hashed array tree — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Hashed_array_tree&oldid=1076479510, 2022. [Online; accessed 25-March-2022]. 本文译自 *arXiv:2211.11009v2* [cs.DS]